

PII DETECTOR AND MASKER

Datasets accidently contain PII

Category: Data Engineering

Submitted by Team 13

V. Anjali

S. Venkata Hemanth Kumar

K. Saisri

S. Sri Ramya

Submitted To

INFINITE COMPUTER SOLUTIONS

June 2026

Abstract:

PII Detector & Masker is a Java-based application developed to identify and protect sensitive information present in datasets. Organizations often handle large volumes of data containing Personally Identifiable Information (PII), such as email addresses, phone numbers, PAN numbers, and Aadhaar numbers. Exposure of such information can lead to privacy violations and security risks.

The proposed system automatically scans CSV and JSON files to detect PII using predefined pattern-matching techniques such as regular expressions. Once identified, the sensitive data is masked to prevent unauthorized disclosure while preserving the usability of the dataset for analysis and testing purposes. The application also generates a detailed report highlighting the types and count of PII detected within the input data.

This project demonstrates the practical application of data privacy principles through automation. Developed using Java and supported by AI-assisted development practices, the solution provides an efficient, reliable, and user-friendly approach to safeguarding confidential information. The system helps organizations reduce manual effort, improve compliance with data protection standards, and minimize the risk of accidental data exposure.

Keywords: Personally Identifiable Information (PII), Data Privacy, Data Masking, Java, Regular Expressions, CSV, JSON, AI-Assisted Development.

Introduction:

In today's digital world, organizations collect and process large amounts of data that often contain Personally Identifiable Information (PII). PII includes sensitive details such as email addresses, phone numbers, PAN numbers, and Aadhaar numbers that can be used to identify individuals. Improper handling or accidental disclosure of such information can lead to privacy breaches, identity theft, and legal consequences.

The PII Detector & Masker is designed to address this challenge by automatically identifying and masking sensitive information present in datasets. The system analyzes CSV and JSON files using predefined pattern-matching techniques to detect various types of PII. Once detected, the confidential data is transformed into a masked format, ensuring that the original information remains protected while preserving the usability of the dataset.

This project provides a practical approach to enhancing data privacy and security through automation. Developed using Java, the proposed solution reduces manual effort, minimizes human error, and enables organizations to handle sensitive data more responsibly. By safeguarding personal information, the system contributes to better data management practices and supports compliance with privacy requirements.

Problem Statement:

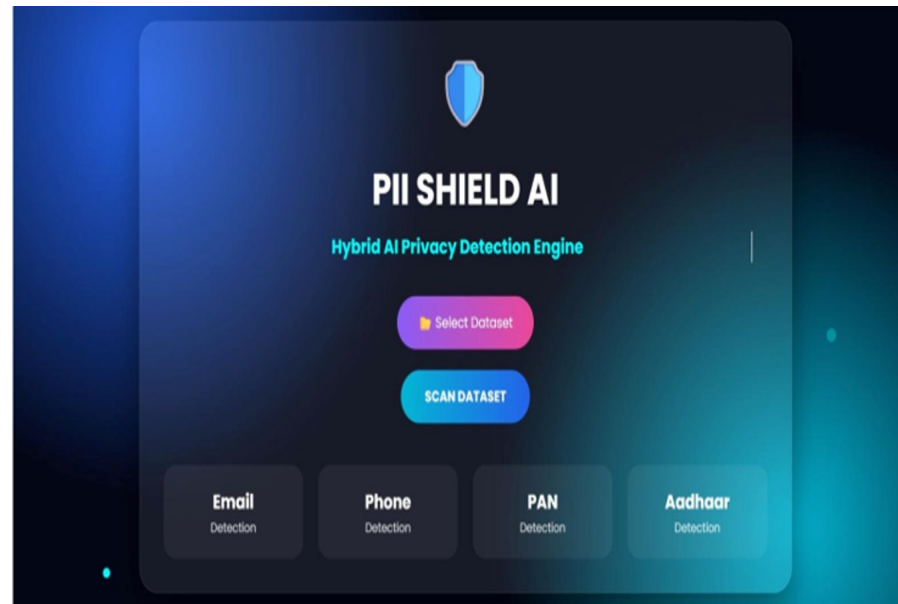
Datasets often contain Personally Identifiable Information (PII) such as email addresses, phone numbers, PAN numbers, and Aadhaar numbers. Manual detection and masking of such sensitive data are time-consuming and prone to errors. Therefore, an automated solution is required to identify and mask PII in datasets to ensure data privacy and security.

Objective:

- To develop a Java-based system for detecting Personally Identifiable Information (PII) in CSV and JSON datasets
- To automatically mask sensitive information such as email addresses, PAN numbers, and Aadhaar numbers.
- To enhance data privacy and reduce the risk of unauthorized disclosure of personal information.

Interface:

With increasing volumes of digital data being processed and shared, the need for secure handling of sensitive information has become essential. The PII Detector & Masker provides a simple interface that allows users to upload CSV or JSON files, detect Personally Identifiable Information (PII), generate masked datasets, and view summary reports efficiently.



Dashboard :

1. Compliance Score

This component provides a real-time health check of the processed data against global privacy regulations such as GDPR, HIPAA, and CCPA. It calculates an aggregate risk rating based on the volume, severity, and types of unmasked PII detected within the system. This score serves as an immediate visual indicator for compliance officers to evaluate data readiness before sharing or storage.

2. Detection Analytics

A dynamic analytics hub that breaks down the specific categories of PII identified across datasets, such as financial identifiers, personal credentials, and medical data. Utilizing interactive charts and distribution graphs, it highlights trends, frequent data leaks, and system-wide vulnerabilities. This granular visibility helps administrators understand what types of sensitive data are entering their ecosystem most frequently.

3. Scan History

A comprehensive, auditable log of all past scanning activities, detailing file names, timestamps, user identities, and total entities detected. It allows administrators to track remediation progress over time, review historical data risks, and maintain a clear paper trail for external regulatory audits. Each entry provides deep-dive access into the specific parameters used during that particular scan session.

4. Download Masked File

The final remediation gateway that allows users to securely export their anonymized datasets in multiple formats like CSV, JSON, or PDF. It ensures that all selected PII is permanently redacted, hashed, or tokenized according to the user's active

configuration profile. The export function strictly adheres to zero-retention principles, downloading the cleaned file directly to the local environment without caching sensitive data on the server.

Backend Framework & Core Language :

- **Python**

Chosen as the foundational programming language for its robust ecosystem, exceptional readability, and premier support for data science and natural language processing. It serves as the core engine driving the application's business logic, data manipulation, and string-parsing workflows.

- **Flask**

A lightweight and flexible WSGI micro-framework used to build the application's backend REST APIs. Flask routing efficiently handles requests from the frontend, manages user sessions, and connects incoming text or file payloads to the underlying PII detection and masking services without unnecessary architectural overhead.

Data Processing & Handling :

- **Pandas**

A powerful, high-performance data manipulation library utilized to ingest, parse, and process structured data files like CSVs and Excel spreadsheets. It enables rapid tabular data cleaning, allows for precise column-by-column PII scanning, and handles complex dataset transformations efficiently.

- **JSON (JavaScript Object Notation)**

Used as the primary data exchange format for seamless, lightweight communication between the Python backend and the JavaScript frontend. It structures configuration rules, scan results, and metadata payloads, ensuring fast and structured data parsing across the entire network architecture.

Frontend Interface :

- **HTML (HyperText Markup Language)**

Provides the semantic backbone and structural layout of the user interface. It defines the layout for essential dashboard components, including the file drag-and-drop zones, metric display cards, side-by-side comparison panes, and configuration forms.

- **CSS (Cascading Style Sheets)**

Governs the visual presentation, styling, and responsiveness of the dashboard layout. It ensures a polished, clean user interface with consistent typography, custom-colored PII entity highlights, and fluid responsive designs tailored for varying screen sizes.

- **JavaScript**

Drives the dynamic, client-side interactivity of the dashboard application. It handles asynchronous API requests (AJAX/Fetch) to send data to the backend without refreshing the page, manages real-time UI updates, manipulates the DOM to display highlighted PII, and triggers the file download flows.

Testing Framework :

- **Pytest**

Utilized as the primary testing framework to write clean, readable, and scalable automation test suites for the backend. It ensures the reliability of the PII detection logic, verifies regex edge-cases, and tests masking functions through structured fixtures and parameterized inputs before deployment

Project Architecture :

1. Upload File

The system entry point where users securely upload unstructured text, CSVs, or PDFs via an asynchronous frontend interface. The backend validates the file format and extracts the raw text data into memory for processing without permanent storage.

2. PII Detection Engine

The core analytical layer that scans the extracted text using predefined regular expressions, pattern matching, and entity recognition. It identifies, classifies, and logs the exact locations of sensitive data points like names, emails, and financial identifiers.

3. Masking Engine

The data transformation layer that applies the user's chosen privacy configuration to the detected PII. It securely replaces sensitive data using techniques such as full redaction, cryptographic hashing, or tokenization while preserving the surrounding text context.

4. Compliance Calculation

A logical assessment engine that analyzes the volume and severity of the remaining unmasked data against global regulations like GDPR and HIPAA. It translates these findings into a standardized risk percentage and compliance health score.

5. Dashboard Analytics

The data visualization layer that aggregates the processing metadata, classification breakdowns, and risk scores into structured JSON format. This structured data is fed directly to the frontend to render real-time, interactive charts and history logs.

6. Masked Output File

The final architecture node where the successfully processed, anonymized data is compiled back into its original file structure. It triggers a secure browser-side download for the user, adhering strictly to a zero-retention data privacy policy.

Folder Structure :

➤ detector

Contains the core PII detection logic, including custom regular expressions, entity recognition algorithms, and classification rules for identifying sensitive data types.

➤ processors

Houses the masking and data transformation scripts responsible for executing redaction, hashing, tokenization, and file formatting operations.

➤ templates

Stores the backend HTML templates that define the structure and view layouts of the web application dashboard.

- static

Holds the client-side assets required for the UI, including CSS files for styling and JavaScript scripts for dynamic page behavior.

- tests

Contains the automated pytest suites used to validate backend logic, edge cases, and API endpoints before deployment.

- uploads

A temporary workspace directory where raw, unmasked files are staged safely during the initial upload and ingestion phase.

- output

The storage directory where fully processed, anonymized files are compiled before being downloaded by the user.

- data

Stores local lookup tables, configuration files, or fallback reference dictionaries used to aid the detection engines.

- web_app.py

The main entry point of the application containing the Flask server configuration, API routing, and request-handling logic.

- README.md

The primary documentation file detailing project overviews, installation guides, dependency lists, and execution instructions.

Installation Guide :

1. Clone Repository

This command downloads a complete and independent copy of the remote project codebase from the hosting platform (such as GitHub or GitLab) onto your local machine. It mirrors the entire directory structure, commit history, and source files into a newly created local directory. Running this step ensures you have the exact, up-to-date files required to build, configure, and execute the application. This acts as the fundamental starting point for setting up your local development environment.

2. Move into Project

This step utilizes the change directory (cd) command to shift your terminal's operational focus directly into the newly downloaded project root folder. Navigating into this specific directory is mandatory as all subsequent configuration, installation, and execution scripts are context-dependent. It ensures that the terminal can properly locate essential configuration items such as the dependencies list, application files, and core subdirectories. Without executing this, system commands will look in the wrong folder location and throw file-not-found errors.

3. Create Virtual Environment

This initializes a self-contained, isolated Python environment specifically allocated for the PII Detector and Masker application. By utilizing Python's built-in venv module, it establishes a distinct sandbox directory that contains its own Python binary and independent package directory. This practice is crucial in modern development to prevent version conflicts between your global computer setup and this project's dependencies. It guarantees that any third-party packages installed later will not disrupt or overwrite other applications running on your system.

4. Activate Environment (Windows)

Running this activation script shifts your terminal's environment path variables to prioritize the local virtual sandbox over your global system configuration. It visually prefixes your command prompt with (venv), giving you immediate confirmation that the isolated workspace is successfully running. While active, any Python scripts executed or packages managed will be strictly confined to this specific project ecosystem. This step must be performed every time you open a new terminal window to work on or run the dashboard application.

5. Install Requirements

This instruction commands Python's package manager (pip) to read the requirements.txt file and automatically download all necessary third-party libraries. It fetches critical frameworks like Flask for the backend API structure and Pandas for high-speed tabular data processing, handling all sub-dependencies seamlessly. By automating this process, it guarantees that every developer or reviewer runs the exact same software versions across different systems. Once this process finishes executing, the backend environment is fully provisioned and ready to safely launch the application.

Running the Application :

1. Run Application

This command executes the main Python script using your terminal's active virtual environment to launch the local web server. It triggers the Flask framework initialization sequence, which compiles the application routes, imports the data processing configurations, and boots up the backend PII engine. The script binds the application to a local network port and opens a continuous stream that listens for incoming client requests. Keeping this terminal window open is mandatory, as closing it will instantly terminate the background processes and shut down the dashboard. It serves as the live engine that bridges your frontend interface requests with the backend masking algorithms.

2. Open Browser

This step requires copying the local hosting URL and pasting it directly into the address bar of any modern web browser. The network address points directly to your computer's loopback interface, acting as a secure bridge to access the running Flask application locally. Once the page loads, the browser interprets the HTML structure, applies the CSS styles, and initializes the JavaScript controllers to render the user dashboard. This interface provides you with immediate, visual access to upload data records, monitor compliance metrics, and run detection algorithms. It establishes a completely isolated, private workspace where you can interactively manage your sensitive documents safely on your machine.

Running Tests :

1. Running Tests

This command triggers the automated testing suite by invoking the pytest framework within your active terminal environment. It automatically searches the directory structure for test files, loads the predefined verification scripts, and executes them against your core codebase. The framework systematically passes mock data through your regular expressions, boundary conditions, and file processors to ensure everything functions perfectly. It provides a quick, automated health check to verify that new code changes or framework updates haven't accidentally broken existing system features.

2. Expected Result

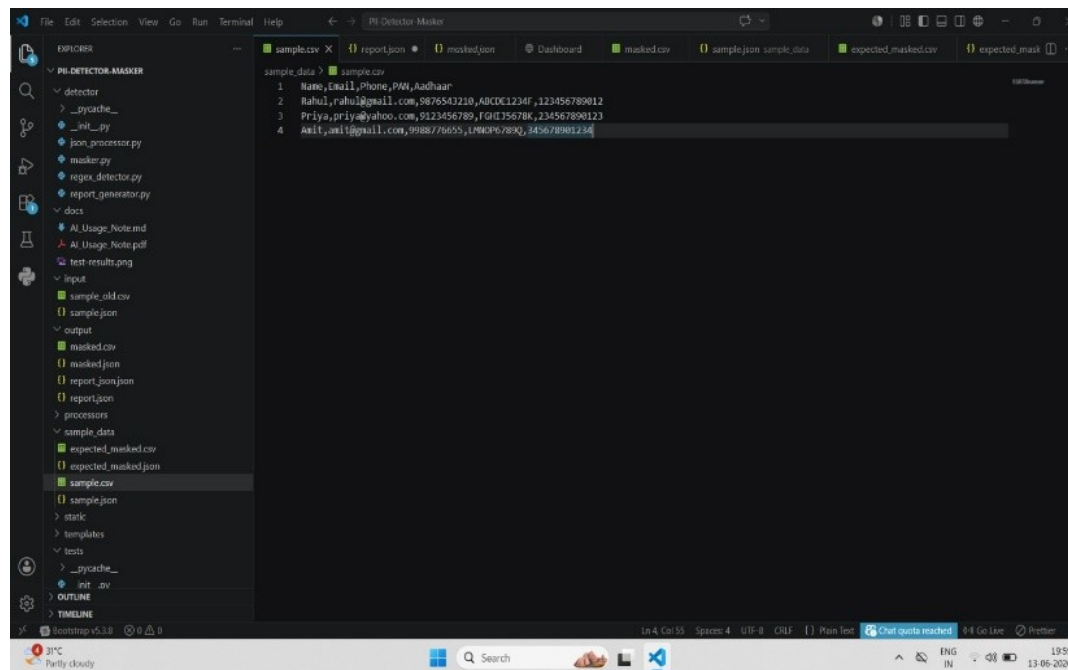
Upon successful completion of the test suite, the terminal will output a green status report indicating that all 10 unit tests passed. This confirms that the PII detection engine accurately identifies sensitive entities, the masking engine redacts data correctly, and the compliance calculator outputs precise scores. A flawless test run guarantees that your edge cases, such as

handling malformed emails or corrupted CSV files, are being managed safely by the backend. It serves as your final quality assurance benchmark, proving the stability, reliability, and security of the application before deployment.

Work flow :

Data Upload:

The user uploads a CSV or JSON dataset containing records that may include Personally Identifiable Information (PII).



Data Reading:

The system reads the uploaded file and extracts all columns and data values for analysis.

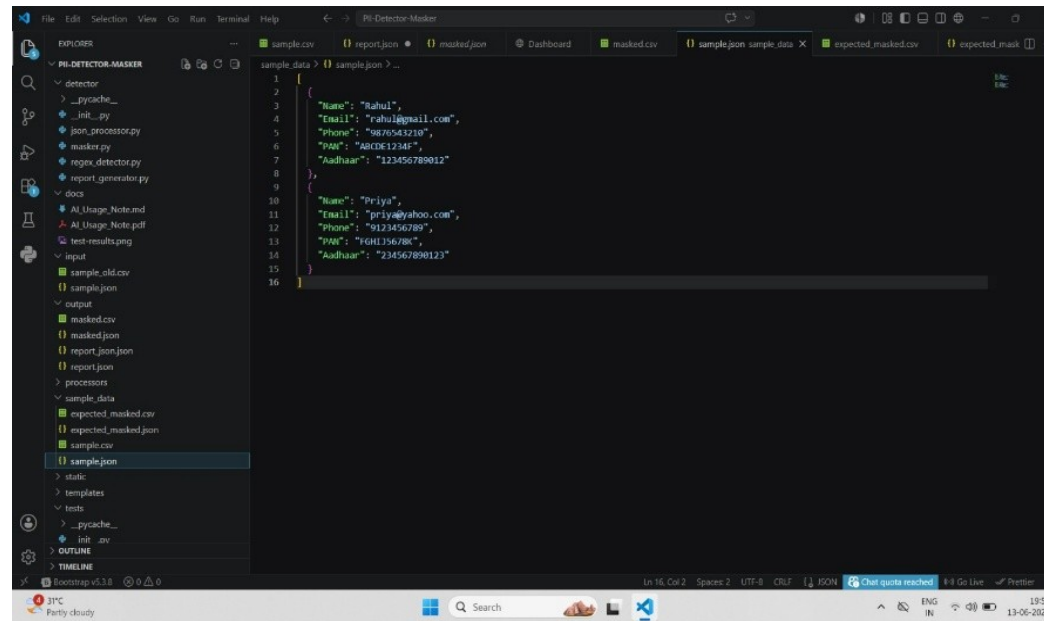
PII Detection:

The detection module scans the dataset using:

- Regular Expressions (Regex) for pattern matching
- LLM (Large Language Model) for contextual identification

The system identifies sensitive information such as:

- Email IDs
- Phone Numbers
- Aadhaar Numbers
- PAN Numbers
- Names (if required)



Classification:

Detected data is classified into different PII categories based on its type.

Data Masking:

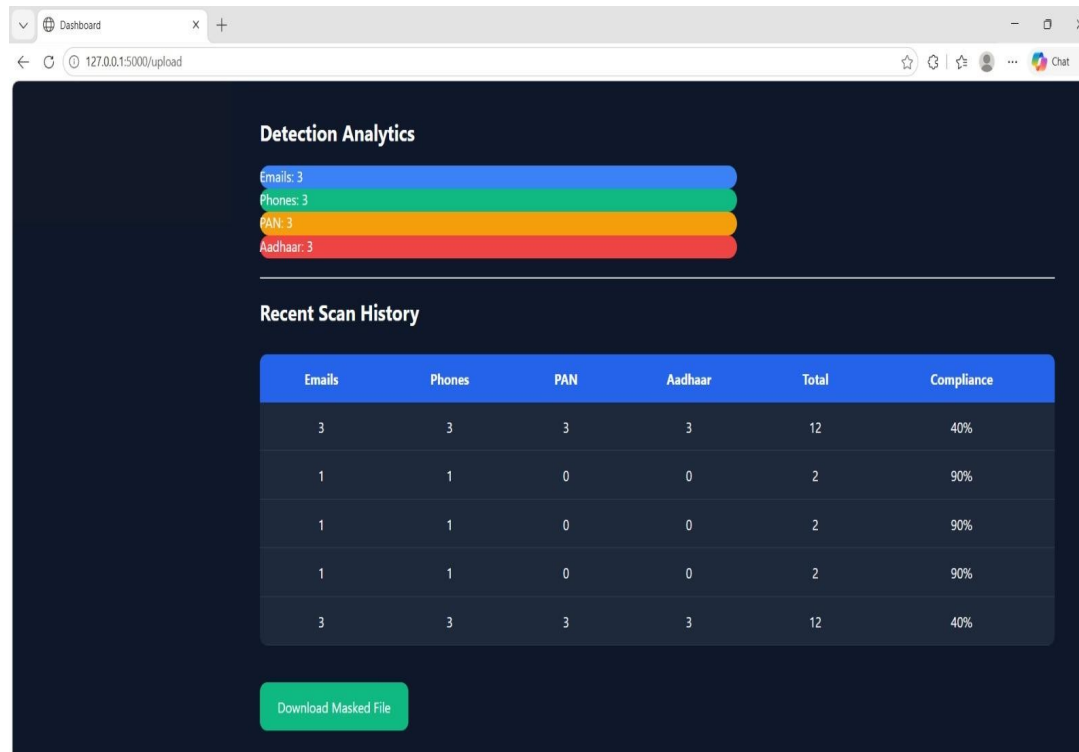
Sensitive information is replaced with masked values to protect privacy.

Examples:

- Email: john@gmail.com → j***@gmail.com
- Phone: 9876543210 → 98*****10
- Aadhaar: 1234 5678 9012 → XXXX XXXX 9012
- PAN: ABCDE1234F → ABCDE****F

Report Generation :

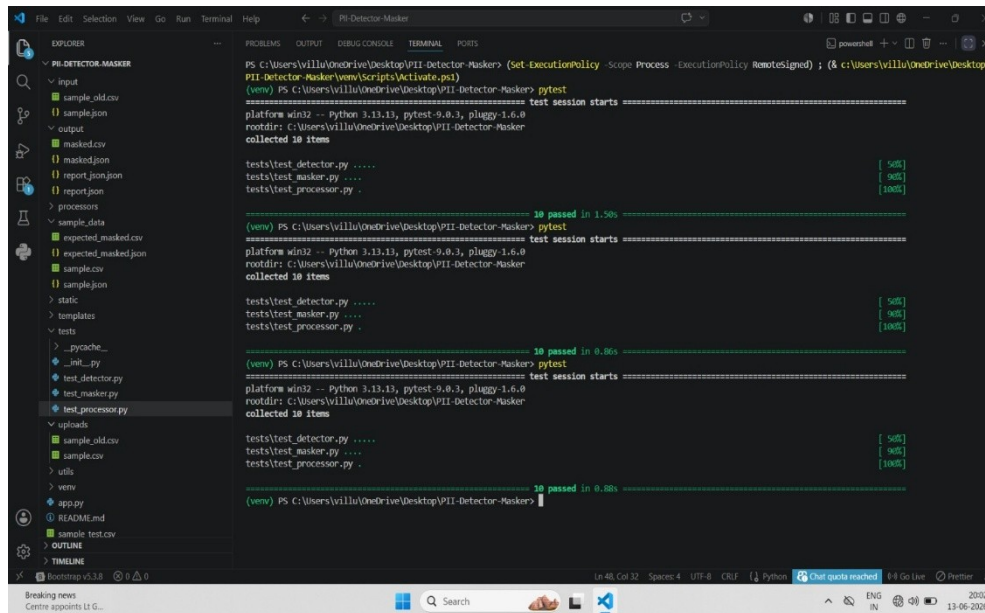
- Total records scanned
- PII types detected
- Number of occurrences
- Masking summary



PyTest:

PyTest is a Python testing framework used to verify that the PII Detector & Masker system functions correctly. It helps ensure that PII detection, masking, file processing, and report generation modules work as expected. Automated testing improves the reliability, accuracy, and quality of the application.

- Objectives of Testing
- Verify correct detection of PII data.
- Validate masking of sensitive information.
- Ensure CSV and JSON files are processed correctly.
- Check report generation accuracy.
- Identify and fix bugs before deployment.



The screenshot shows a Visual Studio Code window with a terminal running PyTest. The terminal output is as follows:

```
PS C:\Users\willu\OneDrive\Desktop\PII-Detector-Masker> (Set-ExecutionPolicy -Scope Process ExecutionPolicy RemoteSigned) ; & c:\Users\willu\OneDrive\Desktop\PII-Detector-Masker\venv\Scripts\Activate.ps1
(venv) PS C:\Users\willu\OneDrive\Desktop\PII-Detector-Masker> pytest
===== test session starts =====
platform win32 -- python 3.13.13, pytest-9.0.3, pluggy-1.6.0
rootdir: c:\Users\willu\OneDrive\Desktop\PII-Detector-Masker
collected 10 items

tests\test_detector.py ..... [ 50%]
tests\test_masker.py ... [ 90%]
tests\test_processor.py . [100%]

===== 10 passed in 1.50s =====

(venv) PS C:\Users\willu\OneDrive\Desktop\PII-Detector-Masker> pytest
===== test session starts =====
platform win32 -- python 3.13.13, pytest-9.0.3, pluggy-1.6.0
rootdir: c:\Users\willu\OneDrive\Desktop\PII-Detector-Masker
collected 10 items

tests\test_detector.py ..... [ 50%]
tests\test_masker.py ... [ 90%]
tests\test_processor.py . [100%]

===== 10 passed in 0.86s =====

(venv) PS C:\Users\willu\OneDrive\Desktop\PII-Detector-Masker> pytest
===== test session starts =====
platform win32 -- python 3.13.13, pytest-9.0.3, pluggy-1.6.0
rootdir: c:\Users\willu\OneDrive\Desktop\PII-Detector-Masker
collected 10 items

tests\test_detector.py ..... [ 50%]
tests\test_masker.py ... [ 90%]
tests\test_processor.py . [100%]

===== 10 passed in 0.88s =====

(venv) PS C:\Users\willu\OneDrive\Desktop\PII-Detector-Masker>
```

The left sidebar shows the project structure with folders like 'input', 'output', 'processors', 'static', 'templates', 'tests', and 'uploads'. The 'tests' folder is expanded, showing 'test_detector.py', 'test_masker.py', and 'test_processor.py'.

Output Generation:

The masked dataset and detection report are saved and made available for download.

Input:

- The system accepts datasets in CSV and JSON formats as input.
- These datasets may contain structured records with personal and sensitive information.
- Once a file is uploaded, the system reads and processes each record to identify Personally Identifiable Information (PII).
- The input data can include fields such as names, email addresses, phone numbers, Aadhaar numbers, PAN numbers, and other confidential details.
- The uploaded dataset serves as the source for the PII detection and masking process.

CSV Format:

Name, Email, Phone, PAN

Ravi, ravi@gmail.com ,

9876543210, ABCDE1234F

JSON Format:

"Name": "Ravi"

"Email": "ravi@gmail.com"

"Phone": "9876543210"

Output:

A. Masked Dataset

Name	Email	Phone	PAN
Ravi	r***@gmail.com	98*****10	ABCDE****F

B. PII Detection Report

PII Type	Count
Email	1
Phone Number	1
PAN	1

Conclusion:

The PII Detector & Masker successfully identifies and protects sensitive information present in datasets. By combining Regex-based pattern matching with LLM-based intelligent detection, the system accurately detects Personally Identifiable Information such as emails, phone numbers, PAN numbers, and Aadhaar numbers. The masking process ensures data privacy while maintaining the usability of the dataset for analysis and sharing. This solution helps organizations comply with data protection regulations and reduces the risk of sensitive information exposure.